

实验任务书：实验十四 系统联调与工程规范

所属课程：《大数据分析》
所属里程碑：M4（数据看板与产品级交付）
实验学时：2 学时
支撑课程目标：目标3（架构思维与前沿技术）、目标4（综合交付与软技能）

一、实验背景与目的

在前几个实验中，我们分别完成了以下几个核心组件：

- 数据清洗与 ELT (Milestone 1):** 在本地利用 DuckDB / Polars 完成了千万级数据的预处理，并生成了列式存储的 Parquet 文件。
- 流批一体数据管道 (Milestone 2):** 编写了实时数据模拟器 (Producer)，利用内存队列/消息队列实现了实时数据消费与机器学习流式预测打标。
- LLM 非结构化数据处理 (Milestone 3):** 调用第三方大语言模型 API 实现了文本细粒度特征抽取（情感、标签、分类等），通过高并发与指数退避重试解决了大规模特征映射瓶颈。
- 数据看板与 Web 交互 (Milestone 4 - Week 12 & 13):** 设计了 FastAPI 数据接口，并利用前端 ECharts 实现了具备多维联动、搜索防抖及高级交互（如维度下钻/刷选）的数据分析看板。

到了第 14 周，我们需要跨越“零散代码片段”到“产品级工程”的鸿沟。

一个成熟的工程项目，仅有能跑的代码是不够的。如果环境难以迁移（依赖缺失或版本冲突）、启动繁琐（需要手动开四五个终端运行不同脚本）、或者遇到环境变动就直接崩溃（路径硬编码、数据库锁死），那么这个系统就无法实现真正的交付。

本实验的训练目标：

- 全链路系统集成 (E2E Integration):** 打通“数据流模拟 -> 队列监听 -> 特征提取 -> 数据持久化 -> 后端 API 服务 -> 前端仪表盘渲染”的整个链条，确保系统整体联调无死锁、无阻塞、高内聚。
- 环境依赖规范化:** 掌握现代 Python 项目的依赖项隔离与精准导出，编写无冗余的 requirements.txt，确保项目在其他环境下的零阻碍重建。
- 交付级文档撰写:** 基于 markdown 标准，撰写具备工业级品质的项目 README.md 文档，利用 Mermaid 绘制清晰的数据流拓扑与系统架构图，确保外部工程师能够快速理解和上手。
- 防御性编程与健壮性调优:** 使用 AI 辅助发现代码中的脆弱环节点，重点处理并发写库冲突（如 DuckDB 并发锁限制）、API Key 缺失、网络抖动等异常，实现优雅退避与降级容错。

二、实验准备

- 代码基础:**
 - 确保前 13 周的所有实验代码和中间数据产出（CSV、Parquet、SQLite 或 DuckDB 数据库文件）均完整保存在工作目录下。
- AI 协同准备:**
 - 准备好 AI 编程助手（如 Claude Code, Qwen Code, Open Code, AntiGravity 等）。
 - 在向 AI 发送指令前，先理清本项目的模块依赖关系与文件目录结构，作为上下文背景提供给 AI。

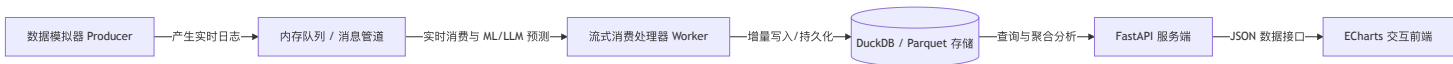
三、实验内容与步骤

任务 1：全链路系统联调与启动脚本设计

在之前的实验中，模拟器、消费者、Web 服务和前端可能分别属于不同的 Python 脚本，启动项目需要打开多个终端手动敲命令。我们需要设计一个一键启动（或最简步骤启动）的方案，并验证数据链路的闭环性。

1. 系统模块盘点与集成拓扑设计

请理清你当前的系统组件。典型的数据流向应满足：



2. 编写集成启动脚本

为了方便交付，我们需要编写一个 Python 启动脚本（例如 `run_app.py`）或者 Shell 脚本（`start.sh`），负责检测依赖并一键拉起后端 API 服务器，必要时自动打开前端浏览器页面。

操作要求：

- 询问 AI 编程助手，编写一个管理子进程的启动脚本 `run_app.py`。
- 功能要求：
 - i. **环境自检**：在启动前，检查当前目录下是否包含必要的数据库文件和前端页面，检测本地 8000 端口（或你设置的 FastAPI 端口）是否被占用。
 - ii. **异步进程管理**：使用 Python 的 `subprocess` 模块异步启动 FastAPI 服务（例如 `uvicorn server:app --reload`）。
 - iii. **自动浏览器唤起**：后端服务启动成功后（可利用 `http` 请求轮询检测端口是否已就绪），使用 Python 的 `webbrowser` 模块自动在默认浏览器中打开前端 `index.html` 的地址（或本地静态托管地址）。
 - iv. **优雅终止**：当用户在命令行按下 `Ctrl+C` 时，脚本能够捕获 `KeyboardInterrupt` 异常，优雅地关闭已创建的后端子进程，不遗留孤儿进程占用端口。

任务 2：环境依赖规范化与可移植性配置

一个常见的低级工程错误是：在全局 Python 环境下运行 `pip freeze > requirements.txt`，将开发机上安装的所有无关包（包括机器学习库的各类型变种、其他项目依赖、IDE 插件依赖等）全部导出。这会导致其他人在新环境中执行 `pip install -r requirements.txt` 时因为依赖冲突或包过大而失败。

操作要求：

1. 清理多余依赖：
 - 绝对不要直接使用全局 `pip freeze`。
 - 推荐使用专门的第三方工具（如 `pipreqs`）分析当前项目目录下的 Python 代码以自动生成最小依赖列表，或者**手动梳理**依赖项。
2. 依赖项版本约束：
 - 编写的 `requirements.txt` 必须仅包含本项目直接 `import` 的核心依赖。例如：

```
fastapi>=0.100.0
uvicorn>=0.22.0
duckdb>=0.9.0
pandas>=2.0.0
polars>=0.19.0
httpx>=0.24.0
scikit-learn>=1.2.0
```
 - 注意：避免写死固定的某个极低或极高的小版本号（如 `==` 可能会在新系统或新 CPU 架构上安装失败），推荐使用安全的大版本范围约束（如 `>=`）。

3. 虚拟环境重建验证：

- 在本地新建一个临时的干净虚拟环境（例如 `python -m venv test_env`），在该环境下激活并执行 `pip install -r requirements.txt`，验证系统是否能无报错、无冲突地完整安装所有依赖。

任务 3：撰写标准项目文档（README.md）

README.md 是任何工程项目交付的“门面”。优秀的文档能够让不了解该项目的人在 3 分钟内理解其业务价值，并在 5 分钟内本地部署成功。

操作要求：

在项目根目录下新建/修改 README.md 文件，文档结构必须包含以下板块：

1. 项目简介与特色 (Project Overview & Features)：

- 明确指出这是一款基于“轻量级高性能数据栈 + 大语言模型 API”的高校课程实验交付项目。
- 简要罗列 3-4 个核心技术特色（如“千万级脱敏日志极速 ETL”、“流式背压管道与 ML/LLM 实时特征预测”、“高并发大模型调用容错设计”、“前后端解耦的动态可视化看板”）。

2. 系统架构与数据流拓扑 (System Architecture)：

- 使用 Markdown 内置的 **Mermaid 语法** 绘制一幅系统的端到端架构图（包含数据采集/模拟、流式处理、模型打标、轻量存储、数据 API 和前端可视化的完整链条）。

3. 快速开始与部署指南 (Quick Start)：

- 详细写明如何从零搭建环境并运行。必须包含：
 - 克隆/下载项目后的虚拟环境创建命令。
 - 依赖项安装命令。
 - 一键运行脚本（如任务 1 编写的 `run_app.py`）的使用方法。

4. 配置说明 (Configurations)：

- 说明如何配置系统变量（例如：如果调用了大模型 API，说明应如何配置 `API_KEY` 或 `.env` 环境变量；如何更改默认的监听端口）。

5. 项目目录树说明 (Directory Tree)：

- 给出项目目录结构的树状图（可使用 `tree` 命令生成或手动整理），并对关键目录和文件做出一句简要解释。

任务 4：防御性编程与系统健壮性优化

在真实运行中，由于网络、数据库锁、文件缺失等原因，系统极易崩溃。请在 AI 编程助手的协助下，对你之前的代码进行一次**代码重构与健壮性提升 (Refactoring for Robustness)**。

操作要求：

1. 处理 DuckDB/SQLite 并发锁限制：

- 现象：**DuckDB 默认不支持多进程同时写入，当流式写入 Worker 正在向数据库 append 数据时，后端 FastAPI 尝试读取可能会报数据库锁死错误（`Database Error: Could not set write lock`）。
- 优化：**在后端 FastAPI 连接 DuckDB 时，配置为只读连接：

```
# 后端服务查询时，只以 read_only=True 形式连接，防止与写入进程冲突
conn = duckdb.connect(database="data/analytics.db", read_only=True)
```

2. 无环境变动崩溃 (Zero-Crash Fallback)：

- 如果系统运行时缺少某个中间数据文件（如 `clean_data.parquet`），不要抛出未捕获的 `Traceback` 直接崩溃。
- 优化：**在代码入口处增加路径检测。如果核心数据文件缺失，自动加载样本数据（或自动运行 M1 中的极简清洗脚本重新生成），并向前端返回友好的 HTTP 错误码（或控制台以亮黄警示输出），提供明确的修复引导。

3. API Key 与外部连接安全卫士（显式降级通知）：

- 设计原则：**针对关键配置（如大模型 API Key）缺失，**严禁静默忽略错误**。虽然系统应通过降级机制保证主体可视化系统仍能启动（不抛出 `KeyError` 崩溃），但必须在控制台与前端界面上**显式告知系统处于降级运行状态**。
- 优化要求：**

- a. **后端检测与控制台告警**：在启动时检测大模型相关的环境变量（如 `SILICONFLOW_API_KEY` 或 `DASHSCOPE_API_KEY` 等）是否配置。若未配置，控制台输出清晰的醒目警告日志（使用 `Python logging.warning` 或带颜色输出），并退避至备用方案（如规则库或轻量词典）。
- b. **前端状态透传与显式提示**：在后端提供一个状态或配置检测接口（如 `/api/system-status`），向前端返回当前系统的运行状态及降级标记（如 `{"llm_active": false, "reason": "API_KEY_MISSING"}`）。前端在收到该降级状态时，需在看板界面显著位置（如顶部）展示提示横幅，明确告知用户“当前大模型功能已降级为内置规则库计算，请配置 API Key 以启用完整功能”。

任务 5：基于 AI 协同的 Git 规范管理与仓库同步

在大数据项目中，由于涉及大量数据集、本地数据库缓存和虚拟环境依赖，良好的版本控制习惯对于团队协作和代码资产管理至关重要。我们将借助 AI 编程助手规范本地项目的 Git 提交历史，并确保在不引入冗余大文件的前提下，将项目安全地同步至 GitHub 仓库。

操作要求：

1. AI 辅助定制精准的 `.gitignore` 规则：

- **痛点**：在之前的实验中，可能会产生几百 MB 的原始 CSV 数据文件（如 `user_behavior_100M.csv`）或 DuckDB 数据库文件（如 `analytics.db`）。如果误将其 `git push` 到 GitHub，会导致仓库超限报错或克隆极其缓慢。
- **操作**：向 AI 描述你的项目目录树，让 AI 编写一个精细化的 `.gitignore` 文件。
- **必须忽略的内容**：
 - 所有大体积的 CSV、Parquet 数据文件（如 `data/*.csv`，`data/*.parquet`，`*.csv`）
 - 本地 DuckDB/SQLite 数据库文件及各种临时锁文件（如 `*.db`，`*.db.tmp`，`*.db.wal`）
 - Python 虚拟环境目录（如 `.venv/`，`env/`，`test_env/`）
 - IDE 缓存与配置文件（如 `.vscode/`，`.idea/`，`.DS_Store`）

2. 使用 AI 辅助生成规范的 **Commit Message**：

- **痛点**：在真实企业工程开发中，应严格避免使用“update”、“fixed”、“111”等无意义的提交日志。
- **操作**：在完成集成联调与代码重构（任务 1-4）后，整理你的修改项（例如“编写了一键启动脚本、清理了依赖包、实现了 API Key 缺失的显式降级与前端横幅提示”），并让 AI 辅助生成一条符合 **Conventional Commits（约定式提交）** 规范的 Git Commit 日志。
- **日志参考格式**：

```
feat(m4): integrate e2e startup script, cleanup dependencies and implement explicit degradation for missing API key
```

3. 推送至 GitHub/Gitee 云端仓库：

- 在本地执行提交后，将其推送（`git push`）到你在 Milestone 1 中创建的 GitHub/Gitee 远程仓库中。
- 检查 GitHub/Gitee 网页端文件目录，确保没有错误提交任何大体积数据文件，且目录树结构清晰。

四、实验报告与验收要求

1. 实验结果与交付物记录

- **项目结构树**：展示你的项目最终目录树（可使用 `tree` 打印），要求结构清晰，无残留的临时垃圾文件。
- **一键启动日志**：截图展示运行一键启动脚本（如 `run_app.py`）的控制台输出，以及自动弹出的前端页面截图。
- **依赖与文档片段展示**：
 - 贴出你整理出的 `requirements.txt` 完整内容。
 - 贴出你编写的 `README.md` 中用 Mermaid 语法编写的系统架构图代码及渲染效果。
- **Git 仓库同步验证**：
 - 附上你的 GitHub/Gitee 远程仓库公开访问链接。
 - 截图展示网页端你的最后一次 Git 提交记录，要求其 Commit Message 必须符合约定式提交（Conventional Commits）规范。

2. 人机协同开发日志（必须手写，不接受 AI 直接生成的空洞文本）

请在实验报告中手写记录以下四个维度的思考与过程：

1. 你的集成 Prompt 词典：

- 在任务 1 的一键启动脚本编写、或任务 4 的防御性编程优化中，贴出你认为**最核心、逻辑最清晰的一条 Prompt**。
- 分析：你在这条 Prompt 中向 AI 描述了哪些上下文（如子进程的关闭机制、DuckDB 只读锁定机制），使得 AI 给出的代码能够完美契合你的现有系统，而没有产生垃圾代码？

2. 系统联调纠错记录（至少 1 个案例）：

- 记录在全链路打通时，你遇到的最令人头疼的一个“黑天鹅 Bug”（例如：“启动了 FastAPI 后，原本能运行的流写入脚本突然报端口占用/数据库锁死”、“Ctrl+C 关闭主脚本后，后台 uvicorn 进程依然活着导致下次启动失败”等）。
- 描述：你是如何通过排查系统进程、网络端口或数据库连接来定位问题的？最终你**如何通过向 AI 提供调试线索，指引 AI 修改代码**解决该问题的？

3. 反思：人在系统工程规范与文档架构中的核心角色：

- 在任务 3 编写 README 时，如果只输入一句“帮我给这个项目写个 README”，AI 往往会生成大量假大空的套话。结合你的实际编写过程，谈一谈：为什么架构图的设计、核心技术亮点的提炼、以及部署步骤的边界条件（如 API Key 缺失时的处理），必须由人类工程师进行主动设计和裁剪，而不能完全交给 AI 自行发挥？

4. Git 与大文件控制反思：

- 描述你如何通过 AI 建立 `.gitignore` 规则来防止大文件误提交的。如果你在开发中不小心把一个 100MB+ 的大体积数据文件执行了 `git commit`（但尚未 `push`），请记录你向 AI 询问的“如何撤销该 commit 并彻底从本地 Git 历史中抹去该大文件”的解决方案和命令。